A yes from someone who could not judge the change is not a control. Send each risk to a check that can catch it.

Friday, November 6, 2026

Thoughtworks named "human-in-the-loop theater" in its new Technology Radar. In the same week, three papers measured how poorly coding agents and AI reviewers judge their own work, and a firm-level study found review is where agent gains stall. This week's daily editions showed coding-agent tools moving approvals into the harness, where a check that cannot decide now says no.

A note on sourcing: arXiv, Thoughtworks, and several blogs could not be opened from this week's research environment, so the summaries below rest on abstracts and search extracts of the primary pages, not full texts. Where coverage disagreed, I say so.

THE STORY OF THE WEEK

Thoughtworks Technology Radar 35 calls out human-in-the-loop theater

Thoughtworks published Technology Radar Vol. 35 on Nov 4, with the headline that as AI agents scale, teams need stronger controls. According to the press release, the question has moved from how fast agents write code to whether teams can trust, review, and afford what agents produce, and it advises teams to assess their review capacity before scaling agent use.

The blip to read is Human-in-the-loop theater: a person kept in an agent's loop as an approval step who approves actions, prompts, or diffs they cannot realistically evaluate. The Radar calls this an accountability sink, a decision that nobody actually owns. Its companion technique, risk-tiered human oversight (Trial), scales review depth to what a change could break: intermediate-risk changes get human review, and low-risk changes can be merged automatically by agents through CI. For changes beyond human judgment, the blip says teams should stop reviewing individual changes and instead steer the system with mutation testing, property-based testing, formal models such as TLA+ and Quint, and agent-assisted semantic review. Coverage such as ByteIota also reports that sandboxed execution moved to Adopt for all agents, not just coding agents. I could not confirm that on the Radar's own pages.

What we can learn: the Radar gives a widely read name to a failure most clients will recognize. A review gate counts the number of approvals, not whether anyone was able to judge what they approved. The practical move is to sort changes by risk and reversibility, decide which check owns each class, and stop asking people to sign off on what only a test, a property, or a model can verify.

> **Takeaway:** Ask a client to list which of its agent-change approvals the approver could genuinely judge. The rest need a named automated check, not a faster reviewer.

THREE READS WORTH YOUR WEEKEND

Critical thinking when agents do the work

Addy Osmani, X Article, early November (Nov 3 per coverage)

Osmani, who works on Claude Code, says his daily work no longer forces critical thinking on him now that agents do much of it, so he has to build the habit on purpose. He describes a range from offloading (you delegate but still own the result), through deferral, to "cognitive surrender", a term from Wharton researchers Steven Shaw and Gideon Nave, where the agent's output silently becomes your conclusion. Repeated surrender builds what he calls cognitive debt.

Most interesting: his ownership test. If you cannot explain the reasoning behind an answer, even after asking the agent, you have given up ownership of the decision. Coverage summarizes his countermeasures as defining "verified" before the work starts, remembering that a passing check proves one property, and using a fresh-context critic for risky changes. I read coverage of the post, not the post itself.

Why it matters: This is the individual version of the Radar's theater warning, and it turns verification debt into habits a team can practice.

Artificial Intelligence in the Firm: Bottlenecks in Software Production

Fiona Chen and James Stratton, Harvard working paper on Jellyfish data (a version dated Oct 31 circulated this week; the paper first appeared earlier in 2026)

A firm-level study of engineering-analytics events (commits, pull requests, and Jira issues) at software companies that adopted AI coding agents. Coverage reports that code per worker rose about 30% after adoption, but finished tasks did not rise by a statistically significant amount. Time from opening to merging a pull request rose 49%, and the share of pull requests with changes requested nearly doubled.

Most interesting: the mechanism. The paper models review as a stage with roughly fixed capacity, so faster coding piles up as backlog and longer waits rather than shipped work. Versions differ on the sample: coverage cites 718 firms and about 300 million events through March 2026, while a later abstract cites a

larger sample. It is observational data from one analytics vendor's customers.

Why it matters: It is the strongest evidence so far that review and verification, not code generation, limit agent-driven delivery. That is the case for investing in automated checks that take load off reviewers.

Release: llm 0.37

Simon Willison, Nov 2

The release notes for Willison's LLM command-line tool open with the complaint many open-source maintainers share about a flood of AI-assisted pull requests. He says LLM is the project of his most affected by it and that he is not finding it to be a problem. A secondary report says the release carries fixes from 12 authors, almost all AI-assisted.

Most interesting: the stance. An AI-assisted pull request becomes a well-documented bug report with an attached example, and the maintainer decides whether to merge it or write a fix. That treats intake as triage rather than review.

Why it matters: Teams that receive agent pull requests from outside the core group, such as other departments, vendors, or internal platforms, can adopt the same stance: the pull request is evidence of a problem, not a change the team has to accept.

FROM THE LAB

Built on Broken Assumptions (BOBA): Why Coding Agents Cannot Be Left Alone

Cuadron, Gupta | preprint, Nov 1

BOBA-Bench strings 20 consecutive requests into each of 280 chains from 140 real open-source projects, and tests ten coding agents. Each request arrives in a fresh conversation on the same codebase, and a separate verifier runs the hidden tests for every earlier request after each submission. Agents complete about 95% of requests in isolation but only 51% in chains. They declare work done once their own tests pass, even when those tests miss a mistake, and requests built on that code fail about twice as often. Asked whether an implementation is correct, agents are right 53% of the time. One external "that's wrong" and one repair round lift chain completion from 51% to 59%.

For practice: An agent's own tests and self-review are not a gate. In multi-step agent work, run an independent check after every step, because an undetected miss is inherited by everything built on top of it.

Do Coding Agents Catch Their Own Bugs? (They Don't.)

Patel, Liao, Henderson, He, Chen | preprint, Oct 31

Foresight-SWE has 212 samples across 4 languages and 35 open-source repositories. Each one pairs a pull request with a later one that fixed a defect the first introduced. The foresight resolve rate counts a success only if the agent solves the original task and also avoids that later defect. Across eleven models, the average resolve rate is about 60.7%, but the foresight resolve rate is about 14.8%. When the defect is described in the task, agents fix it about 91% of the time. The gap is in spotting the defect, not in fixing it. An earlier, smaller version of the benchmark appears in the NeurIPS 2026 listings.

For practice: "Task done, tests green" hides the defects a change creates for the next one. Regression, integration, and property tests written by someone other than the agent are where those defects get caught.

CR-Bench-v2: Evaluating the Real-World Utility of AI Code Review Agents

Pereira, Sinha, Ghosh, Dutta (Nutanix) | preprint, Nov 2

The benchmark turns SWE-Bench issues into pull-request review tasks with full repository context: 584 instances, including a 174-instance subset drawn from SWE-Bench Verified. It scores eight review agents on four model backbones, plus Gemini Code Assist, with three independent LLM judges. Two metrics are kept apart: Known-Defect Detection Rate (did the reviewer flag the labeled defect) and Usefulness Rate (how many comments are actionable). The strongest systems catch about a third of known defects. Gemini Code Assist catches 8.1% while most of its comments rate as useful. Note that the paper body still calls the benchmark CR-Bench, and an earlier CR-Bench paper from March reported different metrics.

For practice: A review bot whose comments developers like can still miss most real defects. Measure AI review tools by what they catch on seeded or known defects, not by satisfaction with their comments.

FROM THE FIELD

An engineer moves from PostHog to an AI-native startup and counts the difference

Neil Kakkar's essay How software engineering is changing: a tale of two eras won The Pragmatic Engineer's fifth-anniversary essay prize ($10,000, from 125 entries). Kakkar compares his years at PostHog with his first eight months as a founding engineer at Tano, an AI-native startup, and reports merging about 11 times as many pull requests a month at Tano. One summary puts his PostHog baseline at 8 to 20 a month.

The operating model is the part to study. Kakkar built a code review agent that runs in CI on every pull request, and in most cases it is the only review. Human review is optional once an engineer knows the codebase. He names the costs: constant context switching, less shared understanding of the code, and trouble keeping focus.

The honest gaps: it is one person's account, pull request counts are easy to inflate, and coverage notes there is no defect, incident, or customer-impact data, and no detail on how much agent work needed correction. I read summaries and excerpts, not the full essay. Set it next to the Radar and the Jellyfish paper: this team removed the review bottleneck by making an agent the default reviewer. CR-Bench-v2 says that reviewer will miss most known

defects unless tests and other checks own them.

THE QA LENS

QA teams are already merging what nobody verified

TestMu AI's State of AI in Quality Engineering 2026 survey, fielded late July to early October to its community of testers, developers, and QA leaders, reports that 81% of teams have merged AI-written code or tests they did not fully understand, 19% of them regularly. 44% report more escaped defects or production incidents as AI-generated code grew, and only 13% have an incident-response plan for AI mistakes. It is a vendor survey of its own community, not a random sample.

Read it next to the lab results. Agents judge their own correctness about as well as a coin flip (BOBA), avoid only about one in seven defects their changes introduce (Foresight-SWE), and AI reviewers catch about a third of known defects (CR-Bench-v2). The fix is not more human sign-off: Thoughtworks calls that theater when the person cannot judge the change. On test maintenance, a study of LLM repair of tests broken by source changes (Chen et al., Nov 2, 500 cases from 15 Java projects) finds that failure messages, the source diff, and supporting code each add context and work best combined.

QA's opportunity is to own the checks between agent steps: an independent regression run after each agent change, mutation or seeded-defect scores for generated tests and AI reviewers, and a short list of change types that still need a person who can actually judge them.

ONE BIG IDEA

> ***A yes from someone who could not judge the change is not a control. Send each risk to a check that can catch it.***

WEEKEND RABBIT HOLE

Run BOBA-Bench against your own coding agent

What it is: the BOBA-Bench repository publishes the benchmark's code, task archives, and Docker images. Each task is a chain of 20 dependent requests on one codebase, with a separate verifier that re-runs the hidden tests for every earlier step after each submission.

Why it was selected: most agent benchmarks score one task at a time. This one scores what happens when a mistake is inherited, which is how agent work actually accumulates in a repository. Running a few chains with and without an external check between steps shows how much your own verification setup recovers.

Pay attention to: the results are the authors' own, with no independent replication yet, and the repository's per-model leaderboard names models I could not confirm. Use it to compare your own configurations, not to rank vendors.

FUTURE BET

BET: By the end of 2027, at least one major AI code review product will publish its defect-detection rate on a public, labeled benchmark next to its comment-usefulness numbers.

WHY: CR-Bench-v2 shows the two numbers can diverge sharply: one commercial reviewer caught 8.1% of known defects while most of its comments rated as useful. Thoughtworks now warns against approvals nobody can judge, which pushes buyers to ask what an automated reviewer actually catches.

SOURCE: Pereira et al. (arXiv 2611.02566), Thoughtworks Technology Radar Vol. 35

Bet check. *By the end of 2027, a major code host will record whether an approval was submitted by a person or by an agent acting through that person's account:* **Holding.** No code host changed its review record this week. This week's harness changes do tie approvals more tightly to context: Codex 0.167.0 documents command rules that require a fresh human approval, and LangGraph 1.2.20 can refuse an approval answer meant for an earlier pause. The Radar's accountability-sink language names the same gap.

RADAR CHECKPOINT

Decision: add **Fail-closed agent approvals** (Governance, Experiment now). In one week, Copilot CLI 1.0.106 required explicit approval for scripts it cannot parse and made its assisted-mode judge fail closed, Codex 0.167.0 documented rules that force a fresh human approval, Claude Code 2.1.317 and the Agent SDK let a failing hook callback block the action, Claude Code 2.1.318 bound a shell approval to the folder it was granted in, and LangGraph 1.2.20 added a strict resume that refuses stale approval answers. The counterexample came from the same week: Copilot CLI now applies blanket-approval flags on resume. The research above shows why it matters: agent self-checks and AI reviewers miss most defects, so the harness has to refuse when it cannot decide. Evidence refreshes without new items: honest non-zero exits after a lost transcript or a cut-off reply join Agent run provenance records, Claude Code's pooled gateway budgets join Agent run budgets, Codex's managed network constraints join Agent reach controls, and BOBA, CR-Bench-v2, and the Radar's human-in-the-loop theater blip join Independent agent quality gates. Human-in-the-loop theater stays a framing inside quality gates rather than its own item, and proactive agents that start work on a rule remain a candidate: one research prototype so far.

View the current SDLC Radar